

Java Collections Framework

Complete Technical eBook

Beginner → Advanced → Expert

Internal Working • Big-O Analysis • Interview Q&A;
HashMap Internals • ConcurrentHashMap • Red-Black Trees
Performance Tuning • Real-World Design Patterns

11 Chapters

50+ Interview Q&A

100+ Code Examples

Performance Tables

CHAPTER 1

Introduction to Java Collections Framework

The Java Collections Framework (JCF) is one of the most fundamental and widely used subsystems of the Java Standard Library. Introduced in Java 1.2 (1998), it provides a unified architecture for storing, retrieving, manipulating, and communicating aggregate data. Before its introduction, Java developers relied on ad-hoc data structures — arrays, Vectors, and Hashtables — each with inconsistent APIs, limited functionality, and no common abstraction.

At its heart, the Collections Framework is an exercise in object-oriented design: it separates **interfaces** (what a collection can do) from **implementations** (how it does it), and provides a rich set of **algorithms** through utility classes. This separation is what makes it so powerful — you can write code against the *List* interface today and swap between *ArrayList* and *LinkedList* tomorrow without touching a single line of business logic.

1.1 Why the Collections Framework Was Introduced

Prior to Java 1.2, the Java language shipped with only three container types: arrays (primitive language constructs), `java.util.Vector` (a resizable array, but with invasive synchronization on every method), and `java.util.Hashtable` (a key-value store, again fully synchronized). These classes suffered from several serious problems:

- **Lack of a common interface.** You could not write a single method that accepted either a `Vector` or an array.
- **No standard algorithms.** Every team wrote their own sort, search, and min-max utilities, leading to code duplication.
- **Poor performance.** `Vector`'s blanket synchronization was unnecessary overhead for single-threaded use-cases.
- **Fixed-size arrays.** Primitive arrays cannot grow once allocated, forcing programmers to manage resizing manually.
- **Type safety.** Without generics (pre-Java 5), collections stored `Object`, requiring casts and risking `ClassCastException` at runtime.

■ Interview Insight: The JCF was designed by Joshua Bloch (author of *Effective Java*) and modelled on the C++ Standard Template Library (STL), but with a cleaner, interface-driven design.

1.2 Limitations of Arrays

Arrays are the most primitive data container in Java. While they offer $O(1)$ random access, they carry significant limitations for general-purpose programming:

- **Fixed size:** Once created with `new int[100]`, the array cannot shrink or grow. You must allocate a new array and copy data.
- **No built-in methods:** Arrays have no `add()`, `remove()`, or `contains()` methods.
- **Homogeneous type only:** An array can hold one type. You cannot mix strategies without object arrays and casting.
- **No iterator protocol:** You must manually manage index-based traversal.
- **Primitive and object gap:** You cannot store primitives in a generic way without boxing overhead.

1.3 Dynamic Resizing — The Core Concept

The most important innovation of collections like `ArrayList` and `HashMap` is **automatic dynamic resizing**. These structures maintain an internal array. When the array becomes full (or crosses a load threshold), the collection allocates a new, larger array and copies all existing elements into it. This operation is $O(n)$ in the worst case but — crucially — happens infrequently enough that the **amortised cost per insertion is $O(1)$** .

```
// Conceptual pseudocode for ArrayList growth
if (size == internalArray.length) {
    int newCapacity = internalArray.length * 3 / 2 + 1; // Java 8
    formula
    internalArray = Arrays.copyOf(internalArray, newCapacity);
}
internalArray[size++] = element;
```

1.4 The Collections Hierarchy

The JCF is built on two parallel hierarchies:

- **Collection hierarchy** — rooted at `java.lang.Iterable`, then `java.util.Collection`, branching into `List`, `Set`, and `Queue`.
- **Map hierarchy** — rooted at `java.util.Map`, which is *not* a subtype of `Collection` because it maps keys to values rather than storing individual elements.

```
Iterable
  ■■■ Collection
    ■■■ List (ArrayList, LinkedList, Vector, Stack)
    ■■■ Set (HashSet, LinkedHashSet, TreeSet)
    ■■■ Queue (PriorityQueue, ArrayDeque, LinkedList)
    ■■■ Deque (ArrayDeque, LinkedList)
  Map [Separate hierarchy]
    ■■■ HashMap
    ■■■ LinkedHashMap
    ■■■ TreeMap
    ■■■ Hashtable
    ■■■ ConcurrentHashMap
```

1.5 Arrays vs Collections — Detailed Comparison

Feature	Arrays	Collections
Size	Fixed at creation	Dynamic resizing
Type safety	Covariant (unsafe)	Generics-based (safe)
Primitives	Supported natively	Autoboxing required
Algorithms	java.util.Arrays only	Rich utility methods
Null support	Yes	Depends on implementation
Performance	Fastest (cache-friendly)	Slight overhead
Thread safety	Not inherent	Optional (concurrent classes)
Iteration	Manual index loop	for-each / Iterator
Memory	Compact	Object header overhead

CHAPTER 2

Core Interfaces of the Collections Framework

The power of the JCF comes from its carefully designed interface hierarchy. Interfaces define contracts — what a collection promises to do — without specifying how. This separation enables polymorphism, testability, and the ability to swap implementations.

2.1 Iterable<E>

`java.lang.Iterable` is the root of all collections that can be traversed. It declares a single method: `Iterator<T> iterator()`. Any class implementing `Iterable` can be used in a Java **enhanced for-loop**. Internally, the compiler transforms:

```
for (String s : list) { ... }

// into equivalent bytecode:
Iterator it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    // body
}
```

This transformation means every `Collection` subtype benefits from the enhanced for-loop automatically, since `Collection` extends `Iterable`.

2.2 Collection<E>

`java.util.Collection` extends `Iterable` and is the foundational interface for all single-element containers. It declares the core CRUD contract:

- `boolean add(E e)` — add one element
- `boolean remove(Object o)` — remove first occurrence
- `boolean contains(Object o)` — membership test
- `int size()` — number of elements
- `void clear()` — remove all elements
- `boolean isEmpty()` — emptiness check
- `Iterator<E> iterator()` — inherited from `Iterable`

Design Principle: `Collection` intentionally does NOT specify ordering or uniqueness — those are concerns of sub-interfaces `List` and `Set` respectively.

2.3 List<E>

`java.util.List` extends `Collection` and adds the concept of **positional access**. Elements are stored in insertion order and can be accessed by index. Key additions include:

```
E get(int index);
E set(int index, E element);
void add(int index, E element);
E remove(int index);
int indexOf(Object o);
List subList(int from, int to);
ListIterator listIterator();
```

Lists allow duplicate elements and maintain insertion order. Implementations differ in their performance trade-offs: `ArrayList` for random access, `LinkedList` for frequent insertion/deletion.

2.4 Set<E>

`java.util.Set` extends `Collection` and adds the uniqueness constraint — no two elements may be equal according to `equals()`. `Set` adds no new methods beyond `Collection`; its contract is enforced behaviourally. Sub-interfaces include `SortedSet` and `NavigableSet` for ordered sets.

2.5 Queue<E> and Deque<E>

`Queue` models a first-in, first-out (FIFO) data structure, adding peek, poll, and offer semantics. `Deque` (double-ended queue) extends `Queue` to support both ends, enabling use as both a queue and a stack.

```
// Queue operations:
offer(e) // add to tail, returns false if full
poll() // remove from head, returns null if empty
peek() // inspect head without removing

// Deque adds:
offerFirst(e) / offerLast(e)
pollFirst() / pollLast()
peekFirst() / peekLast()
```

2.6 Map<K, V>

`java.util.Map` is deliberately not a `Collection` — a `Map` stores key-value pairs, not individual elements. Its hierarchy begins separately and includes `SortedMap` and `NavigableMap`. Core methods:

```
V put(K key, V value);
V get(Object key);
V remove(Object key);
boolean containsKey(Object key);
```

```
boolean containsValue(Object value);  
Set keySet();  
Collection values();  
Set<Entry> entrySet();
```

CHAPTER 3

List Implementations

3.1 ArrayList

ArrayList is the most commonly used List implementation. Internally, it maintains a plain Java **Object array** (`Object[] elementData`). When you create new `ArrayList()` with no arguments, the internal array starts empty and is lazily initialized to capacity 10 on first addition.

Internal Growth Algorithm

```
// From OpenJDK source (simplified)
private void grow(int minCapacity) {
    int oldCapacity = elementData.length;
    // New capacity = old * 1.5 (right shift by 1 = divide by 2)
    int newCapacity = oldCapacity + (oldCapacity >> 1);
    if (newCapacity < minCapacity)
        newCapacity = minCapacity;
    elementData = Arrays.copyOf(elementData, newCapacity);
}
```

The growth factor of 1.5× is a deliberate trade-off between memory waste (a factor of 2 wastes up to 50%) and reallocation frequency (a factor of 1.1 means frequent copies). At 1.5×, amortised insertion is $O(1)$.

Time Complexity

Operation	Best Case	Average Case	Worst Case
<code>add(E)</code> — tail	$O(1)$	$O(1)$ amortised	$O(n)$ — resize
<code>add(int, E)</code> — middle	$O(n)$	$O(n)$	$O(n)$
<code>get(int)</code>	$O(1)$	$O(1)$	$O(1)$
<code>remove(int)</code>	$O(n)$	$O(n)$	$O(n)$
<code>contains(Object)</code>	$O(1)$	$O(n)$	$O(n)$
<code>size()</code>	$O(1)$	$O(1)$	$O(1)$

■ Common Mistake: Calling `remove(element)` inside a for-each loop causes `ConcurrentModificationException`. Always use `Iterator.remove()` or `removeIf(predicate)`.

```
// WRONG — throws ConcurrentModificationException
for (String s : list) {
    if (s.isEmpty()) list.remove(s);
}
```

```
}  
  
// CORRECT – using removeIf (Java 8+)  
list.removeIf(String::isEmpty);  
  
// CORRECT – using Iterator  
Iterator it = list.iterator();  
while (it.hasNext()) {  
    if (it.next().isEmpty()) it.remove();  
}
```

3.2 LinkedList

LinkedList implements both List and Deque. Internally it uses a **doubly-linked list** of Node objects, each holding a reference to the previous node, the next node, and the data element.

```
// Internal Node structure (from OpenJDK)  
private static class Node {  
    E item;  
    Node next;  
    Node prev;  
  
    Node(Node prev, E element, Node next) {  
        this.item = element;  
        this.next = next;  
        this.prev = prev;  
    }  
}
```

Because every node object costs ~48 bytes on a 64-bit JVM (object header + 3 references), LinkedList carries significant memory overhead compared to ArrayList. Random access is $O(n)$ because the list must be traversed from head or tail to reach position i .

When to Use LinkedList

- Frequent insertions/deletions at the **head or tail** ($O(1)$ operation)
- Implementing a queue or double-ended queue (Deque API)
- When you never need random access by index
- When memory overhead of node objects is acceptable

3.3 Vector and Stack

Vector is ArrayList's predecessor, introduced in Java 1.0. It is functionally identical to ArrayList but every method is **synchronized**. This makes it thread-safe but slow in single-threaded scenarios. In modern Java, use `Collections.synchronizedList(new ArrayList<>())` or `CopyOnWriteArrayList` instead.

Stack extends Vector and adds push, pop, and peek semantics. It is considered **legacy**. The preferred alternative is Deque with ArrayDeque as the implementation.

■ Best Practice: Never use Stack or Vector in new code. Use ArrayDeque for stack semantics and ArrayList for resizable lists.

CHAPT
ER 4

Set Implementations

4.1 The Hashing Foundation

All hash-based collections — `HashSet`, `LinkedHashSet`, and `HashMap` — rely on two foundational methods that every Java object inherits from `Object`: `hashCode()` and `equals()`. These form the **hash-equals contract**:

- If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` **must** be true.
- If `a.hashCode() == b.hashCode()`, `a.equals(b)` **may or may not** be true (collision is allowed).
- If you override `equals()`, you **must** override `hashCode()` consistently.

■■ Critical Bug: Overriding `equals()` without overriding `hashCode()` means two logically equal objects can end up in different buckets, so `Set.contains()` will fail to find them.

4.2 HashSet

`HashSet` is backed by a `HashMap` internally. When you add an element to a `HashSet`, it calls `map.put(element, PRESENT)` where `PRESENT` is a static dummy `Object`. Uniqueness is enforced because `HashMap` keys are unique.

```
// From OpenJDK source
private transient HashMap map;
private static final Object PRESENT = new Object();

public boolean add(E e) {
    return map.put(e, PRESENT) == null;
}

public boolean contains(Object o) {
    return map.containsKey(o);
}
```

Insertion order is not preserved. Elements are distributed across hash buckets, so iteration order depends on hash values and appears random.

4.3 LinkedHashSet

`LinkedHashSet` extends `HashSet` and is backed by a `LinkedHashMap`. It adds a doubly-linked list threading through all entries in insertion order. The result is a set that preserves insertion order while still offering $O(1)$ add/remove/contains. The memory overhead is approximately one extra pointer pair per entry.

4.4 TreeSet

TreeSet is backed by a TreeMap, which is itself a **Red-Black Tree**. Elements are stored in sorted order (natural ordering via Comparable, or a supplied Comparator). All basic operations are $O(\log n)$.

TreeSet also implements NavigableSet, providing powerful range-query methods:

```
TreeSet set = new TreeSet<>(List.of(5, 3, 8, 1, 9, 2));

set.first(); // 1 – smallest element
set.last(); // 9 – largest element
set.floor(4); // 3 – greatest element <= 4
set.ceiling(4); // 5 – smallest element >= 4
set.headSet(5); // [1, 2, 3] – elements strictly less than 5
set.tailSet(5); // [5, 8, 9] – elements >= 5
set.subSet(3, 8); // [3, 5] – elements in [3, 8)
```

4.5 Java 8 Treeification in HashSet/HashMap

In Java 8, a critical performance improvement was introduced: when a single hash bucket accumulates **8 or more entries** (a pathological collision scenario), the linked list in that bucket is converted into a **Red-Black Tree**. This changes worst-case lookup within a bucket from $O(n)$ to $O(\log n)$. When entries in that bucket drop to 6 or fewer (due to removals), the tree converts back to a linked list.

```
// Constants in HashMap source

static final int TREEIFY_THRESHOLD = 8; // List -> Tree
static final int UNTREEIFY_THRESHOLD = 6; // Tree -> List
static final int MIN_TREEIFY_CAPACITY = 64; // Min table size
for treeify
```

CHAPTER 5

Map Implementations — Deep Dive

5.1 HashMap — The Most Important Collection

HashMap is arguably the most important and most used data structure in Java enterprise applications. Understanding its internals is essential for both writing efficient code and succeeding in technical interviews. HashMap implements the Map interface using a technique called **hash chaining**.

Internal Data Structure

Internally, HashMap maintains an array of `Node<K,V>[]` called table (the **bucket array**). Each node stores the hash code, the key, the value, and a pointer to the next node (for chaining). The default initial capacity is **16** and the default load factor is **0.75**.

```
// Simplified Node structure
static class Node implements Map.Entry {
    final int hash;
    final K key;
    V value;
    Node next; // for collision chaining
}

// The bucket array
transient Node[] table;
transient int size;
int threshold; // size at which to resize = capacity *
loadFactor
final float loadFactor; // default 0.75f
```

HashMap put() — Step-by-Step Internal Flow

Step 1: Compute `hashCode(key)`. Apply a **secondary hash function** (spread) to redistribute bits: `hash = key.hashCode() ^ (key.hashCode() >>> 16)`. This XOR with the upper 16 bits reduces collisions when the table is small.

Step 2: Compute bucket index: `i = (n - 1) & hash` where `n = table.length`. Since `n` is always a power of 2, this bitwise AND is equivalent to modulo but faster.

Step 3: If `table[i]` is null, create a new Node and store it. Done.

Step 4: If `table[i]` is not null, traverse the chain at bucket `i` looking for a node with equal key (using `==` or `equals()`). If found, replace the value and return the old value.

Step 5: If not found, append a new Node at the end of the chain (or the Red-Black Tree if treeified).

Step 6: After insertion, check if `++size > threshold`. If yes, call `resize()` to double the table and rehash all entries.

```
// Illustrating hash spreading
public final int hashCode() {
    // For String 'Hello':
    // Java's String.hashCode() = 69609650
    // Binary: 00000100 00011001 10010001 10010010
}

static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
    // XOR upper and lower 16 bits to spread entropy
}

// Bucket index calculation
int i = (table.length - 1) & hash(key);
// For table.length=16: (15) & hash => uses only lowest 4 bits
// Spreading reduces collisions on those lowest bits
```

Load Factor and Rehashing

The **load factor** (default 0.75) defines the maximum ratio of entries to capacity before a resize. At 0.75, the HashMap resizes when 75% of buckets are occupied, ensuring reasonable collision probability. A lower load factor (e.g. 0.5) wastes more memory but reduces collisions; a higher value (e.g. 0.9) saves memory but increases collision chains.

During **rehashing**, the table doubles in size (always a power of 2). Each entry is re-distributed: because the new index is $(\text{newCapacity} - 1) \& \text{hash}$ and $\text{newCapacity} = 2 \times \text{oldCapacity}$, each entry either stays at the same index or moves to $\text{oldIndex} + \text{oldCapacity}$. This elegant property means Java 8's rehashing only checks one bit to decide the new position.

5.2 Why HashMap is NOT Thread-Safe

HashMap performs **no synchronization**. In a concurrent environment, two threads calling `put()` simultaneously can corrupt the internal structure in multiple ways:

- **Lost updates:** Both threads read size, both increment it, one write is lost.
- **Broken chains:** During resize, two threads can create circular linked lists (a famous Java bug pre-8, causing infinite loops).
- **Partial visibility:** One thread may see partially constructed Node objects due to instruction reordering without memory barriers.
- **Resize corruption:** If resize happens mid-iteration, the iterator's internal state becomes inconsistent.

5.3 ConcurrentHashMap — Thread-Safe Without Full Locking

ConcurrentHashMap (CHM) is the high-performance, thread-safe alternative to HashMap. Its design has evolved significantly between Java versions:

Java 5–7: **Segment-based locking**. The table was divided into 16 segments, each with its own ReentrantLock. Only the segment being modified was locked, allowing up to 16 concurrent writers.

Java 8+: **Lock-stripping with CAS and synchronized blocks**. Segments were eliminated. The table array itself is the locking structure. Reads are lock-free (using volatile reads). Writes use **Compare-And-Swap (CAS)** for inserting into empty buckets, and synchronized on the bucket head node for non-empty buckets. This allows up to N concurrent writers where N = number of non-colliding buckets.

```
// Simplified ConcurrentHashMap put logic (Java 8)
for (;;) { // spin loop
    Node f = tabAt(tab, i); // volatile read
    if (f == null) {
        // Empty bucket: try CAS to insert atomically
        if (casTabAt(tab, i, null, new Node<>(hash, key, value)))
            break; // CAS succeeded
        // CAS failed – another thread beat us, retry loop
    } else {
        // Non-empty bucket: synchronize on head node only
        synchronized (f) {
            // Safe traversal/insertion within this bucket
        }
    }
}
```

■ Note: ConcurrentHashMap does NOT allow null keys or null values (unlike HashMap). This is intentional: in a concurrent context, a null return from get() would be ambiguous — does it mean key not found, or value is null?

5.4 LinkedHashMap

LinkedHashMap extends HashMap and maintains a doubly-linked list through all entries, recording insertion order. It overrides HashMap's hook methods (afterNodeInsertion, afterNodeAccess, afterNodeRemoval) to update this linked list. The **accessOrder** constructor flag switches from insertion-order to access-order, making LinkedHashMap the foundation for implementing an **LRU (Least Recently Used) cache**.

```
// LRU Cache using LinkedHashMap
public class LRUCache extends LinkedHashMap {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder = true
    }
}
```

```
this.capacity = capacity;
}

@Override
protected boolean removeEldestEntry(Map.Entry eldest) {
    return size() > capacity; // evict when over capacity
}
}
```

5.5 TreeMap

TreeMap implements NavigableMap using a **Red-Black Tree**. All keys are maintained in sorted order. The Red-Black Tree guarantees $O(\log n)$ for put, get, and remove. It provides the richest query API of any Map: firstKey(), lastKey(), floorKey(), ceilingKey(), headMap(), tailMap(), and subMap().

5.6 Red-Black Trees — Internal Working

A Red-Black Tree is a self-balancing Binary Search Tree (BST) where every node is coloured red or black. It maintains these invariants:

- Every node is either red or black.
- The root is always black.
- Every null leaf is considered black.
- A red node's children must both be black (no two consecutive red nodes).
- Every path from a node to its descendant null leaves contains the same number of black nodes (black-height).

These invariants guarantee that the longest path (alternating red-black) is at most twice the shortest (all-black), keeping the tree height bounded at $2 \cdot \log(n+1)$. Insertions and deletions maintain these invariants through **rotations** (left-rotate and right-rotate) and **recolouring**.

```
// Red-Black Tree node (simplified from TreeMap)
static final class Entry implements Map.Entry {
    K key;
    V value;
    Entry left;
    Entry right;
    Entry parent;
    boolean color = BLACK; // new nodes inserted as RED then fixed
                             up
}
```

5.7 Hashtable

Hashtable is the legacy precursor to HashMap, introduced in Java 1.0. Like Vector, every method is synchronized. It does not allow null keys or values. In modern Java, always prefer

ConcurrentHashMap for thread-safe maps and HashMap for single-threaded use.

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable	ConcurrentHashMap
Order	None	Insertion	Sorted	None	None
Null keys	1 null	1 null	No	No	No
Thread-safe	No	No	No	Yes (legacy)	Yes (modern)
Performance	O(1)*	O(1)*	O(log n)	O(1)*	O(1)*
Backing structure	Hash Array	Hash+LinkedList	Red-Black Tree	Hash Array	Hash Array+CAS

CHAPTER 6

Iterators — Traversal and Concurrency

6.1 The Iterator Pattern

The Iterator pattern decouples traversal logic from the collection's internal structure. It provides a uniform interface to traverse any collection regardless of whether it is array-backed, node-backed, or tree-backed. The `Iterator<E>` interface exposes three methods:

```
public interface Iterator {  
    boolean hasNext(); // true if more elements remain  
    E next(); // returns next element, advances cursor  
    default void remove() { // removes last returned element  
        throw new UnsupportedOperationException();  
    }  
}
```

6.2 Fail-Fast vs Fail-Safe Iterators

Fail-fast iterators (used by `ArrayList`, `HashMap`, `HashSet`, etc.) detect structural modifications to the collection during iteration and immediately throw `ConcurrentModificationException`. They achieve this via a `modCount` (modification count) field in the collection. Each structural change increments `modCount`. The iterator records the `modCount` at creation time (`expectedModCount`). On each call to `next()`, it checks whether `modCount == expectedModCount`.

```
// Inside ArrayList.Itr (simplified)  
int cursor; // index of next element to return  
int lastRet = -1; // index of last returned element  
int expectedModCount = modCount; // snapshot at iterator  
creation  
  
public E next() {  
    checkForComodification(); // throws CME if modCount changed  
    // ... return element at cursor, advance cursor  
}  
  
final void checkForComodification() {  
    if (modCount != expectedModCount)  
        throw new ConcurrentModificationException();  
}
```

Fail-safe iterators (used by `CopyOnWriteArrayList`, `ConcurrentHashMap`) operate on a **snapshot** of the collection taken at iterator creation time. Modifications to the collection do not affect the ongoing iteration, and no exception is thrown. The trade-off is that the iterator may not reflect the most recent state of the collection and snapshot creation has a memory cost.

Property	Fail-Fast	Fail-Safe
Examples	<code>ArrayList</code> , <code>HashMap</code> , <code>HashSet</code>	<code>CopyOnWriteArrayList</code> , <code>CHM</code>
Throws CME	Yes	No
Works on	Original collection	Snapshot/clone
Memory overhead	Low	High (snapshot)
Reflects recent changes	Yes (until modified)	No

6.3 ListIterator

`ListIterator` extends `Iterator` and is available only for `List` implementations. It adds bidirectional traversal (`hasPrevious()`, `previous()`), the ability to add elements during iteration (`add()`), and index queries (`nextIndex()`, `previousIndex()`).

6.4 Spliterator

`Spliterator` (Java 8+) is designed for parallel traversal. The name means 'splittable iterator'. It can partition itself into two, allowing the two halves to be processed in parallel. It reports characteristics about the collection (`SIZED`, `ORDERED`, `DISTINCT`, `SORTED`, `IMMUTABLE`, `CONCURRENT`, `NONNULL`, `SUBSIZED`) enabling the Streams API to optimize operations.

CHAPTER 7

Sorting — Comparable, Comparator, and TimSort

7.1 Comparable<T> — Natural Ordering

A class that implements `Comparable<T>` defines its **natural ordering** — the default sort order. The single method `int compareTo(T other)` returns a negative integer, zero, or a positive integer as this is less than, equal to, or greater than other.

```
public class Employee implements Comparable {
    private String name;
    private int salary;

    @Override
    public int compareTo(Employee other) {
        // Natural order: alphabetical by name
        return this.name.compareTo(other.name);
        // Correct way to compare integers (avoids overflow):
        // return Integer.compare(this.salary, other.salary);
    }

    // Usage
    List employees = new ArrayList<>(...);
    Collections.sort(employees); // uses compareTo()
    TreeSet set = new TreeSet<>(employees); // also uses compareTo()
```

■■ Anti-pattern: Never compute `compareTo` by subtraction (`return this.value - other.value`) — integer overflow can produce wrong results for large values. Always use `Integer.compare()`.

7.2 Comparator<T> — External, Flexible Ordering

`Comparator<T>` is a functional interface whose method `int compare(T o1, T o2)` defines an ordering **external** to the class. This enables multiple sort orders for the same type and is essential when you cannot modify the class (e.g. third-party library classes).

```
// Multi-field sort using Comparator chaining (Java 8+)
Comparator comp = Comparator
    .comparingInt(Employee::getSalary) // primary: salary asc
    .thenComparing(Employee::getName) // secondary: name asc
```

```
.thenComparingInt(Employee::getAge) // tertiary: age asc
.reversed(); // flip all to descending

employees.sort(comp);

// Or inline lambda
employees.sort((e1, e2) -> e2.getSalary() - e1.getSalary());
// (Safe here only if salary can't overflow int)
```

7.3 TimSort — The Algorithm Behind Collections.sort()

Java uses **TimSort** for `Collections.sort()` and `Arrays.sort(Object[])`. TimSort was invented by Tim Peters for Python and adopted into Java 7. It is a hybrid of **merge sort** and **binary insertion sort**, specifically optimized for real-world data that often contains already-sorted subsequences (called 'runs').

- **Stable sort:** Equal elements maintain their relative order — critical for multi-key sorting.
- **Adaptive:** Runs as fast as $O(n)$ on already-sorted input, $O(n \log n)$ worst case.
- **Efficient for small arrays:** Uses binary insertion sort for runs smaller than ~64 elements.
- **Memory:** Uses $O(n)$ auxiliary memory for the merge phase.

Note: `Arrays.sort()` for primitive arrays uses Dual-Pivot Quicksort (not TimSort) since stability is irrelevant for primitives and quicksort has better cache performance.

CHAPTER 8

Queue, Deque, and PriorityQueue

8.1 Queue and FIFO Semantics

A Queue models real-world waiting lines — first in, first out. Java's Queue interface provides two sets of operations: throwing versions (add, remove, element) that throw exceptions on failure, and non-throwing versions (offer, poll, peek) that return special values. Always prefer the non-throwing versions in production code.

8.2 PriorityQueue — Heap-Based

PriorityQueue is backed by a **binary min-heap** stored in an array. The element with the lowest priority (as defined by natural order or a Comparator) is always at the head. PriorityQueue does **not** guarantee any order among elements of equal priority.

```
// Min-heap: smallest element at head
PriorityQueue minHeap = new PriorityQueue<>();
minHeap.addAll(List.of(5, 2, 8, 1, 9));
while (!minHeap.isEmpty()) {
    System.out.print(minHeap.poll() + " "); // 1 2 5 8 9
}

// Max-heap using reverseOrder
PriorityQueue maxHeap = new
PriorityQueue<>(Comparator.reverseOrder());
maxHeap.addAll(List.of(5, 2, 8, 1, 9));
System.out.println(maxHeap.peek()); // 9

// Real-world: Top-K frequent elements
PriorityQueue> topK =
new PriorityQueue<>(k, Map.Entry.comparingByValue());
```

Heap operations: **insert** appends at the end and sifts up $O(\log n)$; **poll** removes the root, moves the last element to root, and sifts down $O(\log n)$; **peek** returns root without removal $O(1)$.

8.3 ArrayDeque — Circular Array

ArrayDeque is backed by a **circular array** (ring buffer). It maintains head and tail pointers that wrap around the array, enabling $O(1)$ insertions and removals at both ends without shifting. ArrayDeque is the recommended implementation for both Queue and Stack use cases — it is faster than LinkedList (no node allocation) and faster than Stack (no synchronization).

```
// ArrayDeque as a Stack (LIFO)
Deque stack = new ArrayDeque<>();
stack.push("a"); // addFirst
stack.push("b");
stack.pop(); // removeFirst -> "b"

// ArrayDeque as a Queue (FIFO)
Deque queue = new ArrayDeque<>();
queue.offer("a"); // addLast
queue.offer("b");
queue.poll(); // removeFirst -> "a"
```

CHAPTER 9

Utility Classes — Collections and Arrays

9.1 java.util.Collections

`java.util.Collections` is a utility class containing exclusively static methods that operate on or return collections. It is analogous to `java.util.Arrays` but for collection types.

Key Methods

```
List list = new ArrayList<>(List.of(3, 1, 4, 1, 5, 9, 2, 6));

Collections.sort(list); // TimSort, O(n log n)
Collections.sort(list, Comparator.reverseOrder()); // descending
Collections.reverse(list); // O(n)
Collections.shuffle(list); // random permutation O(n)
Collections.shuffle(list, new Random(42)); // seeded for reproducibility

int idx = Collections.binarySearch(list, 5); // O(log n), list must be sorted

System.out.println(Collections.min(list)); // O(n)
System.out.println(Collections.max(list)); // O(n)

Collections.fill(list, 0); // set all to 0
Collections.copy(dest, src); // dest.size() >= src.size()
Collections.frequency(list, 1); // count occurrences
Collections.disjoint(list1, list2); // true if no common elements
Collections.nCopies(5, "hello"); // [hello, hello, hello, hello, hello]
```

Unmodifiable and Synchronized Wrappers

```
// Unmodifiable – reads allowed, writes throw UnsupportedOperationException
List immutable = Collections.unmodifiableList(list);
Map unmodMap = Collections.unmodifiableMap(map);

// Synchronized wrappers – all methods synchronized on the collection object
List syncList = Collections.synchronizedList(new ArrayList<>());
```

```
// Java 9+ factory methods – truly immutable (no backing mutable collection)
List list9 = List.of("a", "b", "c");
Set set9 = Set.of(1, 2, 3);
Map map9 = Map.of("a", 1, "b", 2);

// Java 10+ – copy factory
List copy = List.copyOf(existingList);
```

■ Key difference: `Collections.unmodifiableList()` returns a view — the underlying list can still be modified through the original reference and the view will reflect changes. `List.of()` returns a truly immutable list with no backing mutable structure.

9.2 java.util.Arrays

```
int[] arr = {3, 1, 4, 1, 5, 9};

Arrays.sort(arr); // Dual-Pivot Quicksort for primitives
Arrays.sort(arr, 1, 4); // Sort subrange [1,4)
int i = Arrays.binarySearch(arr, 5); // O(log n), must be sorted
int[] copy = Arrays.copyOf(arr, 10); // extend with zeros
int[] range = Arrays.copyOfRange(arr, 2, 5); // subarray

Arrays.fill(arr, 0); // set all to 0
System.out.println(Arrays.toString(arr)); // [0, 0, 0, 0, 0, 0]
System.out.println(Arrays.deepToString(matrix)); // 2D array

boolean eq = Arrays.equals(arr1, arr2); // element-wise comparison

// Convert array to List (backed by array – fixed size!)
List asList = Arrays.asList("a", "b", "c");
// asList.add("d"); // throws UnsupportedOperationException
// To get mutable list: new ArrayList<>(Arrays.asList(...))
```

CHAPTER
10

Performance Analysis and Choosing the Right Collection

10.1 Comprehensive Big-O Reference

Collection	Add	Remove	Get/Contains	Iteration	Memory
ArrayList	$O(1)^*$	$O(n)$	$O(1)$	$O(n)$	Low
LinkedList	$O(1)$	$O(1)^{**}$	$O(n)$	$O(n)$	High (nodes)
HashSet	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$	Medium
LinkedHashSet	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$	Medium+
TreeSet	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Medium
HashMap	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$	Medium
LinkedHashMap	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$	Medium+
TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	Medium
PriorityQueue	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	Low
ArrayDeque	$O(1)^*$	$O(1)^*$	—	$O(n)$	Low

* Amortised $O(1)$ — occasional $O(n)$ for resize. ** $O(1)$ at head/tail; $O(n)$ if traversal needed to find element.

10.2 Decision Framework — Which Collection to Use

Selecting the right collection is a critical engineering skill. Use the following decision framework:

Q: Do you need key-value mapping?

→ Use HashMap (no order needed), LinkedHashMap (insertion order), TreeMap (sorted keys).

Q: Do you need element uniqueness?

→ Use HashSet (no order), LinkedHashSet (insertion order), TreeSet (sorted).

Q: Do you need a sequence with duplicates?

→ ArrayList: when random access is frequent.

→ LinkedList: when insertions/deletions at both ends dominate.

→ ArrayDeque: when using as a stack or queue.

Q: Do you need sorted retrieval with range queries?

→ TreeMap or TreeSet — NavigableMap/NavigableSet API enables floor, ceiling, headMap, tailMap.

Q: Do you need thread safety?

→ ConcurrentHashMap for maps.

→ CopyOnWriteArrayList for read-heavy, write-rare lists.

→ Collections.synchronizedXxx() for simple wrapping (iterators need external sync).

→ BlockingQueue implementations (LinkedBlockingQueue, ArrayBlockingQueue) for producer-consumer patterns.

Q: Do you need a priority queue?

→ PriorityQueue for unbounded heap; ArrayBlockingQueue for bounded FIFO.

10.3 Memory Optimization Tips

- **Initialize with capacity:** new ArrayList<>(expectedSize) avoids resizing. For HashMap: new HashMap<>((int)(expectedSize / 0.75) + 1).
- **Use primitive collections:** Libraries like Eclipse Collections or Trove offer IntList, IntHashMap, etc. avoiding boxing overhead.
- **Prefer ArrayDeque over LinkedList:** No per-element Node allocation.
- **Trim to size:** ArrayList.trimToSize() frees unused capacity after bulk loading.
- **Use EnumSet/EnumMap for enum keys:** Backed by bit vectors — extremely compact and fast.

10.4 Performance Tuning Strategies

```
// 1. Pre-size collections to avoid rehashing
Map wordCount = new HashMap<>(1 << 16); // 65536

// 2. EnumSet is the fastest Set for enum types
Set weekdays = EnumSet.range(MONDAY, FRIDAY);

// 3. Use computeIfAbsent for map initialization patterns
Map<> groups = new HashMap<>();
groups.computeIfAbsent(key, k -> new ArrayList<>()).add(value);

// 4. Iterate entrySet() not keySet() when you need both key and value
for (Map.Entry e : map.entrySet()) { // 1 lookup
    use(e.getKey(), e.getValue());
}
// NOT: for (K k : map.keySet()) { V v = map.get(k); } // 2 lookups

// 5. Use getOrDefault() instead of containsKey() + get()
int count = map.getOrDefault(word, 0) + 1;
```

```
map.put(word, count);
```

CHAPTER 11

Top 50 Interview Questions — Beginner to Expert

— BEGINNER —

Q 1: What is the Java Collections Framework?

A: The JCF is a unified architecture of interfaces, implementations, and algorithms for storing and manipulating groups of objects. It was introduced in Java 1.2 and provides List, Set, Map, Queue interfaces with multiple implementations (ArrayList, HashMap, etc.) and utility classes (Collections, Arrays).

Q 2: What is the difference between Collection and Collections?

A: Collection (`java.util.Collection`) is the root interface of the collection hierarchy — it defines the contract for all single-element containers. Collections (`java.util.Collections`) is a utility class with only static methods — `sort()`, `shuffle()`, `min()`, `max()`, `unmodifiableList()`, etc.

Q 3: What is the difference between ArrayList and LinkedList?

A: ArrayList is backed by a dynamic array — $O(1)$ random access, $O(n)$ insert/delete in the middle. LinkedList is backed by a doubly-linked list — $O(n)$ access by index, $O(1)$ insert/delete at head/tail. Prefer ArrayList for most use cases; use LinkedList only as a Deque.

Q 4: What is the difference between List and Set?

A: List allows duplicate elements and maintains insertion order. Set enforces uniqueness (no duplicates). List provides positional access by index; Set does not. HashSet offers $O(1)$ lookup; List.contains() is $O(n)$.

Q 5: What is the difference between HashMap and Hashtable?

A: HashMap is unsynchronized, allows one null key, and is faster. Hashtable is synchronized on every method (thread-safe but slow), does not allow null keys or values, and is considered legacy. Use ConcurrentHashMap instead of Hashtable.

Q 6: What is the difference between fail-fast and fail-safe iterators?

A: Fail-fast iterators (ArrayList, HashMap) throw ConcurrentModificationException if the collection is modified during iteration. Fail-safe iterators (CopyOnWriteArrayList, ConcurrentHashMap) work on a snapshot and never throw CME but may not reflect recent modifications.

Q 7: How does HashSet ensure uniqueness?

A: HashSet is backed by a HashMap. When you add an element, it calls map.put(element, PRESENT). HashMap enforces unique keys, so duplicate elements (as defined by hashCode() and equals()) simply overwrite the PRESENT placeholder rather than adding a second entry.

Q 8: What is the difference between Comparable and Comparator?

A: Comparable defines the natural ordering of a class via compareTo() — the class itself knows how to compare. Comparator defines an external ordering via compare() — useful for multiple sort orders or sorting third-party classes you cannot modify. Comparator is a functional interface in Java 8+.

Q 9: What is an Iterator? How is it different from for-each?

A: Iterator is an interface (hasNext(), next(), remove()) that provides cursor-based traversal. The for-each loop is syntactic sugar that the compiler transforms into an Iterator loop. The key advantage of explicit Iterator usage is the ability to call iterator.remove() safely during traversal.

Q 10: What is the difference between HashMap and TreeMap?

A: HashMap provides O(1) get/put with no ordering. TreeMap provides O(log n) get/put and maintains keys in sorted order (Red-Black Tree). TreeMap also implements NavigableMap with range-query methods. Use HashMap for performance; TreeMap when you need sorted iteration or range queries.

— INTERMEDIATE —**Q 11:** Explain the internal working of HashMap.

A: HashMap uses a Node[] table (bucket array). put() computes hash = hashCode() ^ (hashCode() >>> 16), then index = (n-1) & hash. It chains collisions via linked lists (pre-Java 8) or Red-Black Trees when a bucket exceeds 8 entries (Java 8+). Resizes when size > capacity * loadFactor, doubling the table.

Q
12
:

What happens when two keys have the same hashCode in a HashMap?

A: This is a collision. The new entry is appended to the linked list (or inserted into the Red-Black Tree) at the bucket index. Lookup traverses the chain comparing keys with `equals()`. If the chain length reaches 8 and the table size ≥ 64 , the chain is converted to a Red-Black Tree for $O(\log n)$ worst-case lookup.

Q
13
:

What is the significance of the load factor in HashMap?

A: The load factor (default 0.75) controls when resizing occurs. $\text{threshold} = \text{capacity} * \text{loadFactor}$. When size exceeds threshold, the table doubles and all entries are rehashed. A lower load factor reduces collisions but wastes memory. 0.75 is the empirically optimal trade-off between time and space.

Q
14
:

Why should you override equals() and hashCode() together?

A: The hash-equals contract requires: if `a.equals(b)`, then `a.hashCode() == b.hashCode()`. If you override `equals()` without `hashCode()`, two equal objects may have different hash codes and be placed in different buckets, making `Set.contains()` and `Map.get()` fail to find logically equal objects.

Q
15
:

What is ConcurrentModificationException and how to avoid it?

A: CME is thrown by fail-fast iterators when the collection's `modCount` changes during iteration. Avoid by: (1) using `Iterator.remove()` instead of `Collection.remove()`, (2) using `removeIf(predicate)` (Java 8+), (3) using a concurrent collection like `CopyOnWriteArrayList`, (4) iterating over a copy.

Q
16
:

What is the difference between LinkedHashMap and TreeMap?

A: `LinkedHashMap` maintains insertion order (or access order) via a doubly-linked list through entries — $O(1)$ operations. `TreeMap` maintains sorted key order via a Red-Black Tree — $O(\log n)$ operations but with range-query support. Use `LinkedHashMap` for LRU caches; `TreeMap` for sorted maps with range queries.

Q
17
:

How does ConcurrentHashMap achieve thread safety without locking the whole map?

A: In Java 8+, CHM uses volatile reads for lock-free gets. Puts on empty buckets use CAS (Compare-And-Swap) atomics. Puts on occupied buckets synchronize only on the bucket's head node, allowing concurrent writes to different buckets. No lock is held for reads, and `size()` uses `LongAdder` for accurate counting.

Q
18
:

What is the difference between `poll()` and `remove()` in `Queue`?

A: Both remove the head of the queue. `remove()` throws `NoSuchElementException` if the queue is empty. `poll()` returns null if the queue is empty. Similarly, `peek()` returns null on empty queue; `element()` throws an exception. Always prefer the null-returning versions (`poll`, `peek`, `offer`) in production code.

Q
19
:

How does `PriorityQueue` work internally?

A: `PriorityQueue` uses a binary min-heap stored in an array. The parent of element at index i is at $(i-1)/2$; children are at $2i+1$ and $2i+2$. Insertion appends at the end and sifts up $O(\log n)$. Poll removes the root, moves the last element to root, and sifts down $O(\log n)$. Peek is $O(1)$. No ordering guarantee beyond the minimum at the head.

Q
20
:

Explain the difference between `Vector` and `ArrayList`.

A: Both are backed by dynamic arrays. `Vector` synchronizes every method, making it thread-safe but $\sim 3\times$ slower than `ArrayList` in single-threaded code. `ArrayList` is not synchronized. Both grow when full: `Vector` doubles capacity; `ArrayList` grows by 50%. `Vector` is legacy — use `ArrayList` or `CopyOnWriteArrayList` instead.

— ADVANCED —

Q
21
:

How does Java 8's treeification of `HashMap` buckets work?

A: When a bucket's chain exceeds `TREEIFY_THRESHOLD` (8) entries AND the table size $\geq$ `MIN_TREEIFY_CAPACITY` (64), the linked list is converted to a Red-Black Tree. This improves worst-case lookup within a bucket from $O(n)$ to $O(\log n)$. When entries drop to `UNTREEIFY_THRESHOLD` (6), the tree converts back to a list.

Q
22
:

How does `LinkedHashMap` implement an LRU cache?

A: Construct with `new LinkedHashMap(capacity, 0.75f, true)` — the boolean enables `accessOrder` mode. In this mode, each `get/put` moves the accessed entry to the tail of the doubly-linked list. Override `removeEldestEntry()` to return true when `size() > capacity`. The eldest (head) entry is automatically removed, implementing perfect LRU eviction.

Q
23
:

What are the properties of a Red-Black Tree?

A: RBT is a BST where: every node is red or black; root is black; null leaves are black; red nodes have only black children; every root-to-leaf path has the same black-height. These invariants ensure $\text{height} \leq 2 \cdot \log(n+1)$, guaranteeing $O(\log n)$ for all operations. Insertions/deletions maintain invariants via rotations and recolouring.

Q
24
:

Why does ConcurrentHashMap not allow null keys or values?

A: In a concurrent context, `map.get(key)` returning null is ambiguous — it could mean the key is absent, or the value is explicitly null. There's no thread-safe way to distinguish with a separate `containsKey()` call (another thread could modify the map between the two calls). Disallowing null eliminates this ambiguity.

Q
25
:

Explain Spliterator and its role in parallel streams.

A: Spliterator (Java 8) is a parallel-capable iterator. `trySplit()` splits it into two halves for parallel processing. It reports characteristics (SIZED, SORTED, ORDERED, DISTINCT, etc.) that the Streams framework uses to optimize operations — e.g., a SIZED+SUBSIZED spliterator enables precise work partitioning without needing to count elements.

Q
26
:

How does ArrayDeque's circular buffer work?

A: ArrayDeque maintains a fixed-size array with head and tail integer indices. `addFirst` decrements head (wrapping around with modulo); `addLast` increments tail. When `head == tail`, the array is full and is doubled. Because elements are never shifted, both-end $O(1)$ operations are true, not amortised due to shifting.

Q
27
:

What is the amortised $O(1)$ cost of ArrayList.add()?

A: Though individual additions that trigger resize are $O(n)$, the cost is spread across all additions. With a growth factor of $1.5\times$, after n total insertions the total copy work is $n/1.5 + n/1.5^2 + \dots \approx 3n$. So the total work for n insertions is $O(n)$, making the amortised cost per insertion $O(1)$.

Q
28
:

How does Collections.sort() differ from Arrays.sort()?

A: `Collections.sort()` (and `List.sort()`) uses TimSort — a stable, adaptive $O(n \log n)$ hybrid. `Arrays.sort(Object[])` also uses TimSort. `Arrays.sort(primitive[])` uses Dual-Pivot Quicksort — unstable but faster in practice for primitives due to better cache performance and no need for stability.

Q
29
:

What is the difference between EnumSet and HashSet?

A: EnumSet is a specialized Set for enum types backed by a bit vector (one long per 64 enum constants). All operations are $O(1)$ with no hashing. It is the most memory-efficient and fastest Set possible for enum types. HashSet uses a HashMap internally with object overhead per entry. Always use EnumSet when the element type is an enum.

Q
30
:

How would you implement a thread-safe LRU cache in Java?

A: Use LinkedHashMap with accessOrder=true, wrapped in Collections.synchronizedMap(), and override removeEldestEntry(). For higher concurrency, use ConcurrentHashMap with a ConcurrentLinkedQueue for the LRU order and periodic cleanup, or use Caffeine/Guava Cache libraries which provide highly concurrent LRU caches.

— EXPERT —

Q
31
:

Explain the HashMap infinite loop bug in Java pre-8 under concurrent access.

A: In Java pre-8, during resize, two threads could each begin rehashing the same bucket. The transfer() method reversed the linked list order. With two threads, a circular reference could form (A->B->A), causing an infinite loop on subsequent get(). Java 8 fixed this by maintaining order during transfer and using Red-Black Trees.

Q
32
:

How does HashMap handle hash collisions for keys with hashCode() always returning the same value?

A: All keys hash to the same bucket. Pre-Java 8: $O(n)$ linked list, making all operations $O(n)$. Java 8+: once the chain reaches 8 entries and table size ≥ 64 , the bucket converts to a Red-Black Tree, improving to $O(\log n)$. This is why HashMap performance degrades gracefully even under pathological inputs.

Q
33
:

What is the secondary hash function in HashMap and why is it needed?

A: The spreading function $\text{hash} = h \oplus (h \ggg 16)$ XORs the upper 16 bits into the lower 16 bits. When the table is small (e.g., capacity 16), only the lowest 4 bits of the hash determine the bucket. Without spreading, keys whose hashCode() differs only in high bits would all land in the same bucket. Spreading distributes entropy from high bits into the low bits used for indexing.

Q
34
:

How does ConcurrentHashMap's size() method work in Java 8?

A: size() uses an internal LongAdder (a striped counter). Each update increments a cell in the stripe array determined by thread affinity, reducing contention. size() sums all cells. This is more scalable than an AtomicLong (single CAS point) under high concurrency. The result is an estimate — exact count is only guaranteed at quiescence.

Q
35
:

How would you detect and fix memory leaks in collection-heavy code?

A: Common causes: static Maps/Lists that grow unboundedly; removing objects from collections but forgetting to remove them from related Maps; using non-equals-hashCode-correct objects as Map keys. Fix with: bounded collections (LinkedHashMap + removeEldestEntry), WeakHashMap for cache entries where GC should reclaim them, heap profiling tools (VisualVM, Eclipse MAT) to identify retained collections.

Q
36
:

Compare CopyOnWriteArrayList vs synchronizedList for concurrent use.

A: CopyOnWriteArrayList creates a new array copy on every write — reads are lock-free and fast; writes are expensive. Best for read-heavy, write-rare scenarios. synchronizedList wraps every method in synchronized — reads and writes are both locked, creating a bottleneck. COWAL iterators never throw CME; synchronizedList iterators require external synchronization.

Q
37
:

What are the time complexities for operations on a Red-Black Tree?

A: Search, insert, delete: all $O(\log n)$ guaranteed due to the height bound of $2 \cdot \log(n+1)$. Rotation and recolouring during insert/delete: $O(\log n)$ in the worst case but $O(1)$ amortised rotations per operation. In-order traversal: $O(n)$. Space: $O(n)$ for n entries, with constant overhead per node (key, value, color, parent, left, right pointers).

Q
38
:

How does TreeMap's NavigableMap API work for range queries?

A: TreeMap.subMap(fromKey, toKey) returns a view backed by the original tree — all reads/writes on the view reflect in the original and vice versa. The view enforces the range constraint. This is $O(1)$ to create and $O(\log n)$ for individual operations. headMap(toKey) and tailMap(fromKey) work similarly. The tree structure makes range iteration $O(k + \log n)$ where k is the number of elements in the range.

Q 39 : **Explain the internal structure of LinkedHashMap and how accessOrder works.**

A: LinkedHashMap extends HashMap and adds before/after references to each Entry, forming a doubly-linked list. In insertion-order mode, entries are linked in insertion sequence. In accessOrder mode, afterNodeAccess() unlinks the accessed entry and relinks it at the tail. removeEldestEntry() is called after each put to potentially evict the head (oldest). Iteration traverses the linked list, not the hash table.

Q 40 : **How does Java's TimSort detect and exploit existing runs?**

A: TimSort scans the array for natural 'runs' (ascending or descending sequences). Descending runs are reversed in $O(k)$. Short runs are extended to minRun length (32-64 elements) using binary insertion sort. Runs are pushed onto a stack and merged when they satisfy invariants (ensuring merge costs are $O(n \log n)$ total). This makes TimSort near-linear for partially sorted data.

Q 41 : **What happens internally when HashMap resizes?**

A: resize() creates a new array of double the capacity. Every entry in the old table is re-distributed: for each entry, if $(\text{hash} \ \& \ \text{oldCapacity}) == 0$, it stays at oldIndex; otherwise it moves to $\text{oldIndex} + \text{oldCapacity}$. This works because $\text{newCapacity} = 2 \times \text{oldCapacity}$ means the new index bit is exactly the bit that was zero before. Java 8's resize preserves list order (no reversal), fixing the pre-8 circular reference bug.

Q 42 : **How would you design a collection that is both sorted and allows $O(1)$ lookup?**

A: No standard data structure achieves both. Trade-offs: TreeMap is sorted with $O(\log n)$ lookup. HashMap is $O(1)$ lookup but unordered. A hybrid approach: maintain both a HashMap and a TreeSet in sync. Writes update both structures $O(\log n)$; lookups use HashMap $O(1)$; sorted iteration uses TreeSet. Alternatively, use a skip list (ConcurrentSkipListMap in Java) which provides $O(\log n)$ sorted operations with good concurrent performance.

Q 43 : **What is the difference between WeakHashMap and HashMap?**

A: WeakHashMap holds weak references to its keys. When a key has no other strong references, it becomes eligible for garbage collection. The GC nullifies the weak reference and a ReferenceQueue is used to remove stale entries during subsequent operations. This makes WeakHashMap ideal for caches where entries should be automatically evicted when keys are no longer used elsewhere.

Q 44 : How does `PriorityQueue` handle duplicate elements?

A: `PriorityQueue` allows duplicates. Elements with equal priority are stored according to their heap position — there is no guaranteed ordering among equal-priority elements. `poll()` returns one of the minimum-priority elements but which one among equals is undefined. If strict FIFO among equals is required, use a `Comparator` that breaks ties by insertion order (e.g., using an `AtomicLong` sequence number).

Q 45 : Explain the memory overhead of common collections.

A: `ArrayList`: 16 bytes header + 8 bytes/reference (64-bit). `LinkedList` node: ~48 bytes (header + 3 references). `HashMap` entry: ~48 bytes (header + 4 fields). Object itself: 16 bytes minimum. Boxing overhead: `Integer` = 16 bytes vs `int` = 4 bytes. `EnumSet`: one or two longs for up to 128 enum values — negligible overhead.

Q 46 : When would you use `IdentityHashMap`?

A: `IdentityHashMap` uses reference equality (`==`) instead of `equals()` for key comparison, and `System.identityHashCode()` for hashing. Use cases: serialization frameworks (tracking object identity to avoid infinite loops in circular graphs), proxy caches (where different proxy instances of the same logical object should be separate keys), and graph algorithms.

Q 47 : How does `Collections.unmodifiableList()` differ from `List.copyOf()`?

A: `unmodifiableList()` returns a view — the returned list reflects changes made to the original mutable list. It throws `UnsupportedOperationException` on write attempts, but reads show current state of the original. `List.copyOf()` creates a defensive copy — the returned list is independent and truly immutable. Modifying the original after `copyOf()` has no effect on the copy.

Q 48 : What is the contract of `Map.Entry` and how is it used for efficient iteration?

A: `Map.Entry` represents a key-value pair retrieved from a `Map`'s `entrySet()`. Iterating `entrySet()` is the most efficient way to access both keys and values because each entry already has both — no second lookup is needed. Using `keySet()` iteration and calling `get()` for each key doubles the number of hash lookups.

Q
49
:

How would you implement a Multimap (one key to many values)?

A: Standard Java has no Multimap. Implement as Map> using computeIfAbsent: `map.computeIfAbsent(key, k -> new ArrayList<>()).add(value)`. Or use Guava's Multimap which provides ArrayListMultimap (ArrayList values), HashMultimap (Set values), and TreeMultimap (sorted keys and values) with proper semantics for `size()`, `containsEntry()`, etc.

Q
50
:

What is the performance difference between synchronized HashMap and ConcurrentHashMap?

A: synchronized HashMap (via `Collections.synchronizedMap`) locks the entire map for every read and write — only one thread can access the map at any time. ConcurrentHashMap in Java 8 uses CAS for empty-bucket writes and per-bucket synchronization for non-empty buckets — reads are fully lock-free. Under high concurrency, CHM can be 10-100× faster. CHM also avoids CME during iteration.

CHAPTER ★

Best Practices, Common Mistakes & Summary

Best Practices

- Always program to the interface: `List<String> list = new ArrayList<>()` not `ArrayList<String>`.
- Pre-size collections: `new ArrayList<>(expectedSize)`, `new HashMap<>(size * 4/3 + 1)`.
- Use `computeIfAbsent()`, `getOrDefault()`, `putIfAbsent()` to simplify Map code.
- Iterate `entrySet()` when you need both key and value to avoid double lookups.
- Prefer `ArrayDeque` over `Stack` and `LinkedList` for stack/queue semantics.
- Use `EnumSet` and `EnumMap` for enum-keyed collections — they are the fastest and most memory-efficient.
- Prefer `List.of()`, `Set.of()`, `Map.of()` for immutable collections.
- Use `ConcurrentHashMap`, not `Hashtable` or `synchronizedMap`, for concurrent maps.
- Always override `hashCode()` when overriding `equals()`.
- Never use mutable objects as `HashMap/HashSet` keys — mutating a key after insertion orphans the entry.

Common Mistakes

- Modifying a collection inside a for-each loop — causes `ConcurrentModificationException`.
- Using `==` instead of `equals()` to compare collection elements.
- Forgetting that `Arrays.asList()` returns a fixed-size list — `add()` throws `UnsupportedOperationException`.
- Using `int` subtraction in `compareTo()` — integer overflow produces wrong ordering.
- Using `HashMap` in multithreaded code without synchronization.
- Treating `ConcurrentHashMap`'s `size()` as exact under concurrent modification.
- Storing `null` in collections that do not support it (`TreeSet`, `TreeMap`, `ConcurrentHashMap`).
- Not initializing `HashMap` with expected capacity — causes unnecessary resizing and GC pressure.
- Iterating `keySet()` and calling `get()` for each key — iterate `entrySet()` instead.
- Using `Stack` — it extends `Vector` and is synchronised. Use `ArrayDeque`.

Interview Preparation Summary

Topic	Key Points
-------	------------

HashMap internals	Bucket array + chaining. Hash spreading. Load factor 0.75. Java 8 treeification at 8 entries.
Thread safety	HashMap unsafe. ConcurrentHashMap = CAS + per-bucket sync. Hashtable = legacy full sync.
Iterators	Fail-fast via modCount. Fail-safe via snapshot. CME = structural modification during iteration.
Sorting	TimSort (stable, adaptive). Dual-Pivot Quicksort (primitives). Comparable = natural order. Comparator = external.
Choosing collections	Access pattern, ordering, thread safety, null support, memory budget.
Red-Black Tree	Self-balancing BST. Height $O(\log n)$. Used in TreeMap, TreeSet, and Java 8 HashMap buckets.
LRU Cache	LinkedHashMap + accessOrder=true + removeEldestEntry(). Or ConcurrentHashMap + external LRU tracking.
Performance	ArrayList $O(1)$ access. LinkedList $O(1)$ head/tail. HashMap $O(1)$ average. TreeMap $O(\log n)$.

■ Final Tip: The Java Collections Framework is not just an API — it is a masterclass in object-oriented design. Every architectural decision (interfaces vs classes, fail-fast iterators, load factors, treeification) has a deep reason behind it. Understanding the WHY makes you a better engineer and a stronger interview candidate.